I. Full Preprocessing-Aware Security Experiment

This appendix specifies the experiment summarized in the main text. All service, oracle and state rules below belong to that same experiment.

A. Ideal Public Binding Service

The ideal service $\mathcal{B}$ maintains the binding table UseCtx and provides the read-only interface $\mathsf{ReadBind}(\mathsf{vk}, \mathsf{sid})$, which atomically reads $\mathsf{UseCtx}[\mathsf{vk}, \mathsf{sid}]$ and returns

$$\begin{cases} \mathsf{Unbound}, & \mathsf{UseCtx}[\mathsf{sid}] = \bot, \\ \mathsf{Bound}(\mathsf{ctx}_0), & \mathsf{UseCtx}[\mathsf{sid}] = \mathsf{ctx}_0. \end{cases}$$

Reads are linearizable with token-generation, response and burn transactions and leave the service state unchanged. The first successful honest response transaction records the full, canonically encoded context before releasing the response. This binding remains unchanged after token-state erasure or identifier burning. Records are indexed by $(\mathsf{vk}, \mathsf{sid})$ and retained for as long as signatures under vk may be verified. Only the prescribed state transitions can update these records; this restriction also applies to corrupted signers.

The public read interface returns the full context of a bound identifier without exposing private token state or secret shares. The verifier requires an authenticated, unambiguous read satisfying the service's consistency requirements. If it cannot obtain such a read, it returns 0 rather than treating the identifier as $\mathsf{Unbound}$.

Since a binding cannot change, previously authenticated bound records may be cached. An $\mathsf{Unbound}$ result requires a fresh read, since the identifier may have been bound since an earlier query. Replacing full contexts with digests requires an additional collision-resistance argument to extend the theorem to that implementation.

Let $\mathsf{Vf}_{\mathsf{lat}}$ denote only the canonical-decoding, challenge and norm checks of the original lattice verifier. When these checks pass, let $\mathsf{ctx}^\star$ be its reconstructed context and b_{read} the result of one binding read. The state-assisted verifier accepts exactly when

$$\mathsf{Vf}_{\mathsf{lat}}(\mathsf{vk}, \mathsf{msg}, \sigma) = 1,$$
$$b_{\mathsf{read}} = \mathsf{Unbound} \quad \text{or} \quad b_{\mathsf{read}} = \mathsf{Bound}(\mathsf{ctx}^\star).$$

The binding check compares the signature context with the service state at the read's linearization point. Since this permits both unbound identifiers and matching contexts with incomplete response collections, unforgeability in these cases still relies on the lattice assumptions.

B. Adversarial Model

The model uses two independent random oracles:

$$\mathcal{O}_{H_{\mathsf{label}}} : \{0,1\}^* \to R_q,$$
$$\mathcal{O}_{H_{\mathsf{ch}}} : \{0,1\}^* \to \mathcal{C}_{\mathsf{ch}}.$$

The challenger implements each oracle by lazy sampling. A first query obtains an independent uniform output in the corresponding range, and a repeated query receives the recorded output. The experiments expose ordinary random oracles with classical access.

The adversary is a two-stage PPT algorithm $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$. Before key generation, it runs

$$(\mathcal{C}, \mathsf{st}_{\mathcal{A}}) \leftarrow \mathcal{A}_0(1^\lambda, t, n, a_{\max}).$$

The experiment returns 0 unless $\mathcal{C} \subseteq [n]$ and $|\mathcal{C}| < t$. Set $\mathcal{H} = [n] \setminus \mathcal{C}$. The challenger then runs

$$(\mathsf{vk}, \{\mathsf{sk}_j\}_{j\in[n]}) \leftarrow \mathsf{KGen}(1^\lambda, t, n, a_{\max}).$$

$\mathcal{A}_1$ receives $\mathsf{st}_{\mathcal{A}}$, vk and $\{\mathsf{sk}_j\}_{j\in\mathcal{C}}$ and may adaptively interleave the two random-oracle interfaces with the following oracles:

1) $\mathcal{O}_{\mathsf{Token}}(j, \mathsf{sid})$ creates, records and returns the public token of an honest signer. The first successful query for an unseen identifier initializes it as fresh; later token generation is permitted only while it remains fresh, and a repeated query for the same pair returns the recorded token.
2) $\mathcal{O}_{\mathsf{Resp}}(j, \mathsf{sid}, \mathsf{act}, \mathsf{Tok}_{\mathsf{act}}, \mathsf{msg})$ first checks structural validity and $j \in \mathcal{H} \cap \mathsf{act}$, then locates the shared record and an unconsumed state $\mathsf{st}_{j,\mathsf{sid}}$ under a fresh or activated identifier. It checks that the entry indexed by j equals its recorded token, computes C_{act} and c, and requires the resulting context to equal the recorded context if already activated. On success, it atomically binds a fresh identifier, consumes the signer's private state and releases at most one newly generated honest response for (j, sid).
3) $\mathcal{O}_{\mathsf{Bind}}(\mathsf{sid})$ returns $\mathsf{ReadBind}(\mathsf{vk}, \mathsf{sid})$.
4) $\mathcal{O}_{\mathsf{Burn}}(\mathsf{sid})$ burns the identifier and erases all remaining honest private states associated with it.

The adversary can locally generate the tokens and responses of corrupted signers using their secret keys. Structural validity checks concern canonical encoding, types, ranges, signer membership, and the domain and entry format of the token mapping. They do not include equality with a recorded token or consistency with an aggregate, challenge or bound context. A structurally invalid request returns $\bot$ without changing state. A response request under an unseen or burned identifier, or for which no live, unconsumed honest signer state exists, likewise returns $\bot$ without changing state. Once such a state has been located under a fresh or activated identifier, a failed recorded-token or context-consistency check, or any response-generation failure, atomically burns the identifier and erases all remaining honest private state before returning $\bot$. These rules also govern aggregate and challenge consistency checks in $\mathsf{ShareSign}$. Token queries are idempotent while the identifier is fresh and are rejected after activation or burning. The burned state is terminal, while previously published tokens remain public.

The challenger maintains

$$\mathsf{SidState}[\mathsf{sid}] \in \{\mathsf{unseen}, \mathsf{fresh}, \mathsf{activated}, \mathsf{burned}\},$$

$\mathsf{UseCtx}[\mathsf{sid}] \in \{\bot\} \cup \{(\mathsf{sid}, \mathsf{act}, C_{\mathsf{act}}, \mathsf{msg})\}$, and $\mathsf{RespSet}[\mathsf{ctx}] \subseteq \mathcal{H}$. The experiment fixes one verification key, so the table notation suppresses it. In a multi-key

implementation, every identifier table and private-state record is indexed by $(\mathsf{vk}, \mathsf{sid})$. Initially, $\mathsf{SidState}[\mathsf{sid}] = \mathsf{unseen}$ and $\mathsf{UseCtx}[\mathsf{sid}] = \perp$ for every identifier, and $\mathsf{RespSet}[\mathsf{ctx}] = \varnothing$ for every context.

The first successful honest response request atomically changes a fresh identifier to activated and records its complete context. Further honest signers may respond under the same identifier only for that recorded context, and each pair (j, sid) produces at most one response. A first successful response also erases every honest private state under that identifier whose signer is outside the selected active set. A recorded-token or context-consistency failure after locating a live state, a response-generation failure, or a burn request moves the identifier to burned and erases every unconsumed honest private state. Producing one honest response does not by itself burn the identifier. After a successful response, the challenger inserts j into $\mathsf{RespSet}[\mathsf{UseCtx}[\mathsf{sid}]]$.

These tables store the state shared by the token and response oracles and by honest executions of $\mathsf{TokenGen}$ and $\mathsf{ShareSign}$. Token creation and retrieval are atomic. To issue a response, $\mathsf{ShareSign}$ checks the state, establishes or checks the context binding, consumes the signer's one-time state and releases the response in a single atomic transaction.

C. Terminal Protocol

Eventually, $\mathcal{A}_1$ returns one pair $(\mathsf{msg}^\star, \sigma^\star)$. Both winning events are evaluated on this same terminal output and in the same execution. If $\mathsf{Vf}^{\mathcal{B}}(\mathsf{vk}, \mathsf{msg}^\star, \sigma^\star) = 1$, let $(\mathsf{sid}^\star, \mathsf{act}^\star, C^\star)$ denote the identifier, active set and aggregate value reconstructed internally by verification from the canonical signature encoding. If canonical decoding fails or verification rejects, these values are undefined and neither winning event can occur. All probabilities are taken over key and token generation, the two random oracles, honest protocol executions and $\mathcal{A}$.

The model assumes static corruption, reliable erasure and a trusted, linearizable binding service with authenticated public reads, immutable bindings and rollback protection. The experiments exclude adaptive corruption, exposure of honest signers' private states and corruption of the service. Once $\mathcal{A}$ produces its terminal output, the protocol state is frozen, and the winning predicates and final verification are evaluated against the same values of UseCtx and $\mathsf{RespSet}$.

For $\mathsf{ctx} = (\mathsf{sid}, \mathsf{act}, C_{\mathsf{act}}, \mathsf{msg})$, define

$$\mathsf{Authorized}(\mathsf{ctx}) = 1 \iff \mathcal{H} \cap \mathsf{act} \subseteq \mathsf{RespSet}[\mathsf{ctx}].$$

Because act is the exact contributing set, the remaining shares can be generated locally by the corrupted signers. Under the same final-state convention, define

$$\mathsf{Complete}(\mathsf{sid}) = 1 \iff \exists \mathsf{ctx} : \ \mathsf{sid}(\mathsf{ctx}) = \mathsf{sid}, \ \mathsf{Authorized}(\mathsf{ctx}) = 1.$$

The quantifier ranges only over recorded contexts. Completion means that all honest responses required by such a context have been released. Once true, it remains true after state erasure or identifier burning.

D. Security Notions

Adaptive-Message Security. The adversary may obtain public tokens before choosing its target message. It wins if it outputs $(\mathsf{msg}^\star, \sigma^\star)$ such that $\mathsf{Vf}^{\mathcal{B}}(\mathsf{vk}, \mathsf{msg}^\star, \sigma^\star) = 1$ and there is no recorded context ctx with

$$\mathsf{Authorized}(\mathsf{ctx}) = 1, \quad \mathsf{msg}(\mathsf{ctx}) = \mathsf{msg}^\star.$$

Partial-response queries make a message nonfresh only when they complete the honest-response requirements of some recorded context for that message.

One-Time-Token Security. Suppose verification accepts $(\mathsf{msg}^\star, \sigma^\star)$ and internally reconstructs $(\mathsf{sid}^\star, \mathsf{act}^\star, C^\star)$. Define $\mathsf{ctx}^\star = (\mathsf{sid}^\star, \mathsf{act}^\star, C^\star, \mathsf{msg}^\star)$ and $\mathsf{ctx}_0 = \mathsf{UseCtx}[\mathsf{sid}^\star]$. The one-time-token winning event occurs if

$$\mathsf{ctx}_0 \neq \perp, \qquad \mathsf{ctx}_0 \neq \mathsf{ctx}^\star, \quad \mathsf{RespSet}[\mathsf{ctx}_0] \neq \varnothing, \quad \mathsf{Vf}^{\mathcal{B}}(\mathsf{vk}, \mathsf{msg}^\star, \sigma^\star) = 1.$$

If canonical decoding fails or verification rejects, the event does not occur. Protection begins as soon as one honest response exposes information derived from the private state; a complete authorized transcript for ctx_0 is not required.

Preprocessing-Aware Unforgeability. The adversary may arbitrarily interleave all permitted oracle queries. Let $\mathsf{Win}_{\mathsf{AM}}$ and $\mathsf{Win}_{\mathsf{OT}}$ denote the adaptive-message forgery and incompatible identifier-reuse events, respectively. It wins the preprocessing-aware experiment if

$$\mathsf{Win}_{\mathsf{PA}} = \mathsf{Win}_{\mathsf{AM}} \vee \mathsf{Win}_{\mathsf{OT}}.$$

These events use $\mathsf{Vf}^{\mathcal{B}}$, with the PA advantage also denoted by $\mathrm{Adv}^{\mathsf{PA},\mathcal{B}}_{\Pi_{\mathsf{pp}},\mathcal{A}}$. The binding check provided by $\mathcal{B}$ rules out the incompatible-context event.

For $X \in \{\mathsf{AM}, \mathsf{OT}, \mathsf{PA}\}$, define

$$\mathrm{Adv}^{X}_{\Pi_{\mathsf{pp}},\mathcal{A}}(\lambda) = \Pr[\mathsf{Win}_X].$$

The corresponding notion holds if this advantage is negligible for every PPT adversary satisfying the corruption bound. The AM branch is ordinary, non-strong, message-based unforgeability: once any context for a message is fully authorized, a second accepting signature on that message is not an AM win. The OT branch separately excludes accepting reuse of an activated identifier under an incompatible context. Both branches are evaluated on the same terminal signature.